Results in Applied Mathematics 30 (2026) 100707

Contents lists available at ScienceDirect

Results in Applied Mathematics

journal homepage: www.elsevier.com/locate/results-in-applied-mathematics

A reliable method for finding intersections of 2D NURBS curves

Yuqiao Gai [a], Gangyi Li [a], Na Lei [a], Wei Dong [b], Jinsan Cheng [c,*]

[a] Dalian University of Technology, China
[b] China Academy of Building Research, China
[c] State Key Laboratory of Mathematical Sciences, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, China

ARTICLE INFO

Keywords:
NURBS
Spline intersection
Hodograph

ABSTRACT

A reliable method is presented to compute the intersection of two-dimensional NURBS curves. Two innovative verification criteria are introduced: an intersection existence criterion based on geometric method, and an intersection uniqueness verification criterion founded on Hodograph theory. The combination of these criteria enables precise identification of parametric subintervals containing a unique intersection point. Experimental results demonstrate that compared to traditional Bézier clipping algorithms and piecewise conversion methods, our algorithm significantly improves the computational efficiency and numerical stability of NURBS curves intersection calculations. The method can find intersections in non-singular cases and shows superior robustness, particularly in scenarios with high-degree curves and dense intersections. We also discuss how to deal with singular cases stably. This approach provides both theoretical foundation and practical tools for high-precision intersection computation in complex geometric modeling, path planning, and engineering simulation applications.

1. Introduction

Advancements in computer-aided design (CAD), computer graphics, robotics, medical imaging, and geographic information systems (GIS) have made the intersection of 2D spline curves a critical technology for precise modeling and computation. In practical applications, complex shapes and boundaries often require continuous and smooth curves. Spline curves are widely used due to their flexibility in surface representation [1]. However, unlike linear primitives, spline curves lack simple algebraic solutions for intersection, necessitating iterative methods that must rigorously balance computational efficiency with high-precision convergence. These intersection points are essential for precise model construction, geometric relationship determination, and data visualization. As Hoffmann noted, calculating intersections of parametric curves and surfaces is a foundational challenge in computer-aided geometric design (CAGD) and geometric modeling [2]. Solving intersection points between parameterized curves is not only critical to this problem but also a prerequisite for high-precision geometric computations and complex shape analysis. The accuracy and efficiency of this process directly affect subsequent geometric modeling, shape optimization, and engineering applications, underscoring its theoretical and practical research value [3,4].

In spline curves, Bézier curves [1], defined by Bernstein basis functions and control points, have become a fundamental curve representation method due to their parametric simplicity and geometric intuitiveness. B-spline curves [5], a generalization of Bézier curves, achieve localized control via knot vectors and piecewise polynomial basis functions, providing global smoothness and flexible shape manipulation. These features make B-splines widely used in geometric modeling, animation, and interpolation tasks [6,7]. NURBS (Non-Uniform Rational B-Splines) [8] extend B-splines into the rational domain by incorporating weighted control points,

* Corresponding author.
E-mail address: jcheng@amss.ac.cn (J. Cheng).

https://doi.org/10.1016/j.rinam.2026.100707
Received 10 February 2026; Received in revised form 20 April 2026; Accepted 25 April 2026
Available online 30 April 2026
2590-0374/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

enabling precise adjustments to curve shapes. NURBS can accurately represent conic sections and complex freeform surfaces, establishing them as the standard geometric tool in CAD, computer-aided geometric design (CAGD), and engineering design [9].

Extensive research on spline curve intersections has been conducted globally. The intersection problem of curves is closely related to polynomial root-finding, prompting researchers to study both concurrently. Subdivision-based root isolation techniques have been widely investigated for solving nonlinear systems and polynomial equations. Xu and Yap [10] proposed an effective subdivision algorithm, called Miranda, for isolating the simple zeros of nonlinear functions with interval arithmetic and provided a rigorous complexity analysis. Similarly, Cheng [11] developed a certified numerical method for real root isolation of bivariate nonlinear systems, utilizing local geometric analysis to ensure robustness and efficiency, particularly for high-degree sparse polynomial systems. Building upon these general root-finding techniques, researchers have proposed numerous specialized algorithms tailored for curve intersection problems. Tan [12] presents a novel method for computing intersections of planar parametric curves by reducing and comparing the x and y coordinate ranges of the two curves. Limaiem and Trochu [13] present a geometric approach for intersecting planar parametric curves by employing successive orthogonal projections that ensure convergence even in tangent cases. The Bézier clipping algorithm, first proposed by Sederberg [14], enables efficient computation of intersection points between two Bézier curves and facilitates rapid calculation of their tangent points. Its convergence rate has been shown to be quadratic [15]. Subsequently, various enhanced algorithms based on the Bézier clipping method have been developed to address challenges in polynomial root-finding and curve intersection. For instance, Jüttler [16] and Liu [17] employed degree-reduction techniques to generate quadratic and cubic polynomials, enclosing the polynomial's graph within a valid interval to compute all roots of univariate polynomial equations. Additionally, cubic clipping has been shown to have at least quadratic convergence and has been used to calculate the intersection points of two Bézier curves [18]. Despite this, the Bézier clipping method still requires significant computational resources when applied to higher-degree curves, which can result in slower overall processing speeds. Furthermore, the Bézier clipping method may lead to missed solutions in the presence of multiple roots or when applied to curves of higher degree. North [19] proposed a geometric interval algorithm that tightly bounds curves or contains points on the curve. However, this method is highly sensitive to initial values and computational errors, where even a slight perturbation in the initial guess can substantially alter the calculation results. Yuan [20] made further advancements based on previous work by developing a cubic hybrid clipping method for solving univariate polynomial equations, which achieves a fourth-order convergence rate. This method was later applied by Wu [21] to address the curve intersection problem, where they introduced a technique called cubic hybrid clipping. Through the selection of moving control points for cubic hybrid curves, they achieve better cubic polynomial bounds, thereby enhancing the efficiency of the proposed method. However, due to its complex mathematical computations and the selection of control points, the method is less efficient in simpler cases.

The work of the aforementioned scholars primarily focuses on algorithms for the intersection of two-dimensional Bézier curves without extending to other types of spline curves. Shao [22] and Yu [23] studied the problem of identifying all intersection points between Bézier curves and the B-spline curves. Their algorithm integrates Bézier clipping, the Krawczyk method, and symbolic computation based on Sturm's theorem. This comprehensive approach has demonstrated high efficiency and accuracy in solving algebraic curve intersection problems. However, Sturm's theorem has limitations when applied to intersection problems involving NURBS curves and cannot be used directly. In current engineering practices, piecewise Bézier strategies are commonly employed for NURBS intersection; however, this method has inherent limitations in terms of computational efficiency and precision, underscoring the urgent need for theoretical research on intersection algorithms specifically tailored to the parametric characteristics of NURBS.

To address these challenges, we propose a novel algorithm that overcomes the limitations of existing methods. This method takes two NURBS curves as input and finds all intersections through Bounding Volume Hierarchy (BVH) construction, intersection existence determination, and tangent cone computation.

We advance the field through four key innovations:

- **Criterion of intersection existence :** We propose an innovative sufficient condition for determining whether two curves have an intersection within a specified region.
- **Criterion of intersection uniqueness based on tangent cones :** Based on Hodograph theory, we introduce another sufficient condition to ascertain that the intersection of the two curves within the region consists of at most a single point.
- **Algorithmic completeness for non-singular cases :** The algorithm can theoretically find all intersections in all non-singular cases. Cases like tangency, cusp intersections, and endpoint intersections are considered singular.
- **Reliable and efficient implementation :** We propose a robust algorithm for intersecting 2D NURBS curves, which can find reliable intersections. Experiments show it is up to 10 times faster than classical and open-source methods while ensuring correctness.

The structure of this paper is organized as follows. Section 2 reviews the fundamental tools and essential theorems employed in our algorithm. In Section 3, we present our primary methodology for computing intersection points between two 2D NURBS curves. Specifically, we first outline the complete process of the proposed algorithm. Second, we utilize our intersection existence verification method to confirm the presence of intersection points within specified regions and employ the hodograph to verify the uniqueness of intersections within these regions. Then, we verify the algorithm's completeness for non-singular cases. Finally, we introduce the algorithm's management of singular cases. In Section 4, we validate our proposed method using a variety of test cases, demonstrating the robustness and effectiveness of our algorithm and presenting comparisons with classical approaches. In Section 5, we discuss the practical applications of our algorithm, specifically focusing on its implementation in Boolean operations for 2D closed curves and its utility in surface trimming processes. Finally, Section 6 concludes the paper with a summary of our primary research findings and contributions.

2. Preliminaries

2.1. NURBS

NURBS [8] represents a fundamental mathematical tool for describing free-form curves and surfaces in Computer Aided Geometric Design and computer graphics. By integrating the flexibility of B-splines, the geometric significance of weight parameters, and the adaptability of non-uniform knot distributions, NURBS enables a unified representation of conic sections, elementary analytic surfaces, and complex geometric shapes. A NURBS curve of degree p is defined as:

$$\mathbf{C}(u) = \frac{\sum_{i=0}^{n} N_i^p(u)\,\boldsymbol{w}_i \boldsymbol{P}_i}{\sum_{i=0}^{n} N_i^p(u)\,\boldsymbol{w}_i}, \quad a \le u \le b.$$

Where $P_i \in \mathbb{R}^d$ denotes the control points(d is dimension), $\boldsymbol{w}_i$ is the weight, N_i^p represents the B-spline basis functions of degree p defined over a non-periodic and non-uniform knot vector U, where

$$U = \{a, \dots, a, u_{p+1}, \dots, u_{m-p-1}, b, \dots, b\}.$$

Generally, $a = 0$ and $b = 1$, and for all i, we have $\boldsymbol{w}_i > 0$.

Spline basis function N_i^p generated by the Cox–de Boor recurrence formula [24]:

$$N_i^0(u) = \begin{cases} 1, & \text{if } u_i \le u < u_{i+1} \\ 0, & \text{otherwise,} \end{cases}$$

$$N_i^p(u) = \frac{u - u_i}{u_{i+p} - u_i} N_i^{p-1}(u) + \frac{u_{i+p+1} - u}{u_{i+p+1} - u_{i+1}} N_{i+1}^{p-1}(u).$$

The nonuniformity of NURBS is reflected in the fact that the knot spacing allows for unequal distribution, thus supporting local shape control [25].

Theorem 1. *The number of intersections between any straight line and the NURBS does not exceed the number of intersections between that line and its control polygon [26].*

2.2. Bounding volume hierarchy

A BVH is a tree-structured spatial partitioning method constructed for a set of geometric objects. In this structure, the leaf nodes contain geometric objects that are encapsulated within bounding volumes. Subsequently, these nodes are grouped into smaller sets and enclosed within larger bounding volumes. This process is recursively repeated, with the grouped nodes further grouped and enclosed within even larger bounding volumes, ultimately forming a tree-like structure with a single, overarching bounding volume at its apex. As a spatial data structure, the BVH effectively supports a variety of operations on collections of geometric objects. Typical applications include collision detection, ray tracing, and other tasks in computational geometry [27].

A variety of bounding volumes are commonly used in BVHs, each with distinct characteristics and applications. The Axis-Aligned Bounding Box (AABB) is a widely used bounding volume due to its simplicity and computational efficiency, defined by its minimum and maximum coordinates along each axis and not rotating with the enclosed object. In contrast, the Oriented Bounding Box (OBB) can be rotated to better fit the orientation of the enclosed object, providing tighter bounds than AABBs for arbitrarily oriented objects at the cost of increased computational complexity. Sphere bounding volumes enclose objects within a spherical region defined by a center point and a radius, offering computational efficiency in overlap tests but potentially less tight fits for non-spherical objects. Convex hulls are the tightest possible bounding volumes, formed by connecting the outermost points of the object, and are highly effective for precise collision detection despite their higher computational cost.

In our algorithm, we construct Oriented Bounding Boxes to identify regions between the two curves that definitely do not contain intersection points, thereby enabling the exclusion of these non-intersecting regions.

2.3. Hodograph

The Hodograph [28] is a mathematical tool for representing derivative directions of curves or surfaces, extensively utilized in computer-aided geometric design and computer graphics. Its primary function lies in representing the directional derivatives of curves or surfaces without involving their magnitudes, thereby simplifying computations and offering enhanced efficiency.

Hodographs can be applied to rational Bézier curves and surfaces [29]. These curves and surfaces, due to the introduction of weights, exhibit greater flexibility than polynomial curves and enable more precise descriptions of complex geometric shapes. However, this enhancement introduces new challenges. For rational curves or surfaces, their derivatives are themselves rational curves or surfaces of a higher degree than the original. To improve computational efficiency, researchers have introduced the concept of scaled Hodographs, which represent only the derivative directions while disregarding their magnitudes. When the parameter is u, the Hodograph of the curve is defined as:

$$H(u) = \frac{1}{\boldsymbol{w}^2(u)} \sum_{k}^{2p-2} N_k^{2p-2} H_k,$$

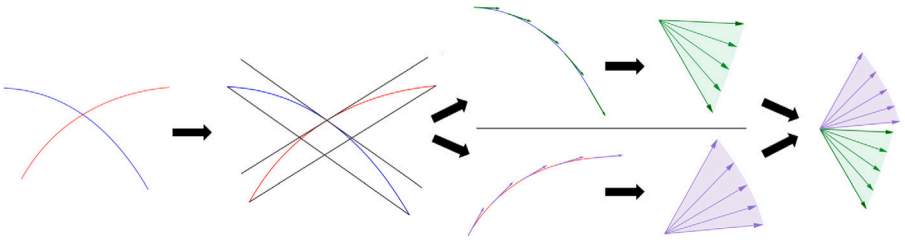


Fig. 1. Existence and uniqueness judgment process. The blue curve represents C_1, the red curve represents C_2, and the black lines are parallel lines. First, construct parallel lines to check for intersections. Next, compute the derivatives of each curve to obtain their tangent cones. Finally, determine the uniqueness of the intersections by comparing whether their tangent cones overlap.

where

$$H_k = \sum_{i=\max(0,k-p)}^{\lfloor k/2 \rfloor} \frac{(k-2i+1)\binom{p}{i}\binom{p}{k-i+1}}{\binom{2p-2}{k}} Dir(P_i, P_{k-i+1}),$$

$$Dir(P_1, P_2) = \boldsymbol{w}_1 \boldsymbol{w}_2 (P_2 - P_1), 0 \le k \le 2p-2.$$

When representing the derivatives of rational curves, these scaled Hodographs are of a lower degree than the original curves, significantly simplifying computations.

In our algorithm, we construct the tangent cone of NURBS curves to determine the uniqueness of intersection points within specified regions.

3. Algorithm

This section presents a reliable method for finding intersections of 2D NURBS curves. The algorithm first checks if the BVH of the two curves intersect to rule out non-intersecting regions. For regions not ruled out, it then evaluates the existence and uniqueness of intersections. For details on evaluating the existence and uniqueness of curves, refer to Fig. 1. After verifying the existence and uniqueness of the intersection points in the specified region, we employ the Newton iteration method to determine the precise intersection points. The midpoint of the curve's parameter range is selected as the initial point for Newton's iteration. If Newton's iteration fails, we modify the initial point and restart the solving process, since we have already confirmed the existence of intersection points within this region. If either the existence condition or the uniqueness condition isn't met, the algorithm performs a subdivision depth check to determine whether the case is singular. The maximum subdivision depth is typically set to six. If not a singular case, the curves will be subdivided, with the subdivision point defaulting to the midpoint of the parameter range [30], and the existence and uniqueness conditions are reevaluated.

The framework of the whole algorithm is summarized in Fig. 2. The "Construct BVH" step will be detailed in Section 3.1. The "Meet the existence conditions?" and "Meet the uniqueness conditions?" steps mentioned in the pipeline will be detailed in Sections 3.2 and 3.3, respectively. The "Special solutions detected" step will be detailed in Section 3.4.

3.1. Construct BVH

To construct a BVH for 2D spline curves, we employ an OBB derived from the control points of the curves. Given a spline curve with control points $\{P_i\}_{i=0}^{n}$, we first compute the primary axis vector $\boldsymbol{v}$ as:

$$\boldsymbol{v} = P_n - P_0.$$

The perpendicular vector $\boldsymbol{v}^{\perp}$ is obtained via a 90° rotation:

$$\boldsymbol{v}^{\perp} = (-v_y, v_x).$$

These vectors define the OBB's local coordinate frame $\boldsymbol{R} = \left[\frac{\boldsymbol{v}}{\|\boldsymbol{v}\|}, \frac{\boldsymbol{v}^{\perp}}{\|\boldsymbol{v}^{\perp}\|}\right]$. Each control point P_i is projected into the local frame $\boldsymbol{R}$:

$$P_i' = \boldsymbol{R}^T (P_i - P_0).$$

The axis-aligned bounding box (AABB) in the rotated frame is computed as:

$$\text{OBB} = \left[\min_x(P'), \max_x(P')\right] \times \left[\min_y(P'), \max_y(P')\right].$$

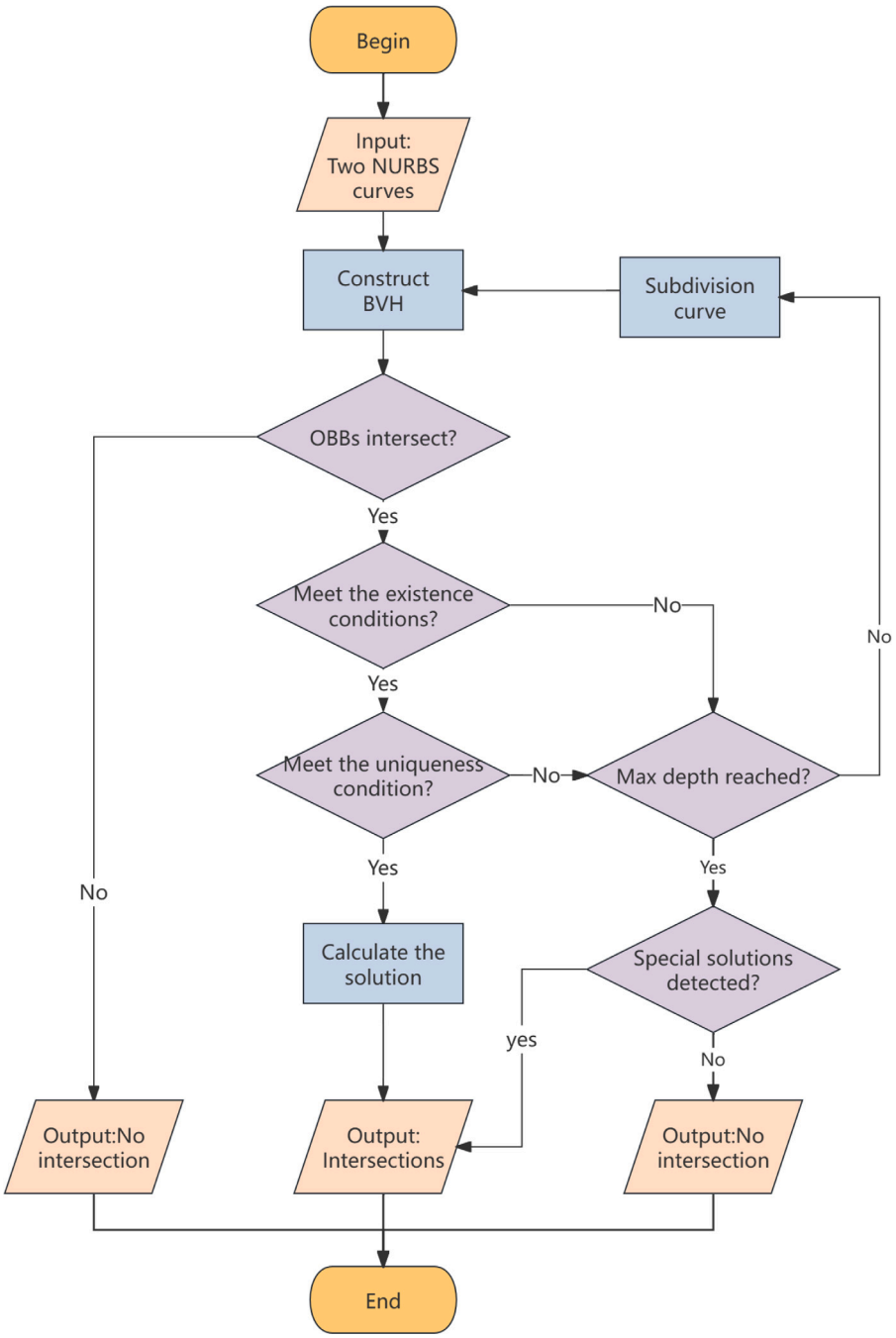


Fig. 2. The pipeline of the reliable method for finding intersections of 2D NURBS curves.

Given two OBBs defined by their centers $\boldsymbol{c}_1, \boldsymbol{c}_2$, half-extents $\boldsymbol{h}_1, \boldsymbol{h}_2$, and rotation matrices $\boldsymbol{R}_1, \boldsymbol{R}_2$, the separation axis theorem (SAT) is applied. For each candidate axis $\boldsymbol{a}$ (columns of $\boldsymbol{R}_1$ and $\boldsymbol{R}_2$), the projected separation must satisfy:

$$|\boldsymbol{a} \cdot (\boldsymbol{c}_2 - \boldsymbol{c}_1)| > \sum_{k=1}^{2} \boldsymbol{a} \cdot \boldsymbol{R}_k \cdot \boldsymbol{h}_k$$

where $|\boldsymbol{a} \cdot \boldsymbol{R}_k|$ denotes the absolute dot product with each column of $\boldsymbol{R}_k$. If no such axis exists, the OBBs intersect. We ensure tight-fitting OBBs for spline curves and efficient intersection queries in the BVH traversal.

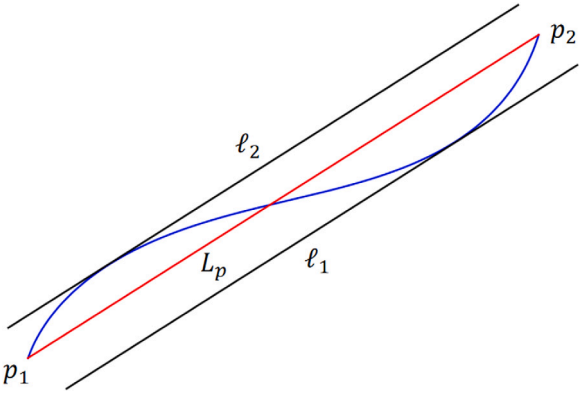


Fig. 3. Schematic diagram of compute support lines. The blue curve is *Curve*, the red line is L_p, the black lines are parallel line.

3.2. Criterion for the existence of an intersection point

Definition 1. Let NURBS curves $C_1(u) : [a_1, b_1] \to \mathbb{R}^2$ and $C_2(v) : [a_2, b_2] \to \mathbb{R}^2$ have endpoints $C_1(a_1) = p_1$, $C_1(b_1) = p_2$, $C_2(a_2) = q_1$, and $C_2(b_2) = q_2$. Define line segments:

$$L_p = \overline{p_1 p_2}, L_q = \overline{q_1 q_2}.$$

Furthermore, construct tightly supporting parallel lines ℓ_1, ℓ_2 parallel to L_p, satisfying:

1. c_1 is completely located in the area between ℓ_1 and ℓ_2;
2. Minimize the width of the area, specifically:

$$Width(\ell_1, \ell_2) = \max_{u\in[0,1]} d(C_1(u), L_p) - \min_{u\in[0,1]} d(C_1(u), L_p),$$

where $d(p_1, \ell_1)$ denotes the sign distance function from point p_1 to line ℓ_1; $Width(\ell_1, \ell_2)$ represents the distance between the lines ℓ_1 and ℓ_2; and the region between ℓ_1 and ℓ_2 is denoted as $Region(\ell_1, \ell_2)$. Similarly, we define tightly supporting parallel lines ℓ_3, ℓ_4 parallel to L_q to constrain C_2 (see Fig. 3 for an illustration).

We express the process of constructing these tightly supporting parallel lines as a function $\ell_1, \ell_2 = ComputeSupportLines(Curve\ C)$.

Definition 2. Let $\{P_i\}_{i=0}^n$ be the control points of the curve $C(u)$. The curve is *monotonic with respect to the line* L if the orthogonal projections of all control points onto L satisfy one of the following conditions:

1. Non-decreasing monotonicity:

$$\pi_L(P_{i+1}) \geq \pi_L(P_i), \quad \forall i \in \{0, \ldots, n-1\}$$

2. Non-increasing monotonicity:

$$\pi_L(P_{i+1}) \leq \pi_L(P_i), \quad \forall i \in \{0, \ldots, n-1\}$$

where $\pi_L(P)$ denotes the scalar projection of the point P onto the direction of L.

Theorem 2. *There is at least one intersection point between the curves C_1 and C_2 if the following conditions are simultaneously satisfied:*

1. *C_1 is monotonic along the direction of L_p, and C_2 is monotonic along the direction of L_q (See Definition 1 and Fig. 3).*
2. *L_p and L_q intersect at a point $o \in \mathbb{R}^2$.*
3. *The points p_1 and p_2 lie on opposite sides of the lines ℓ_3 and ℓ_4, specifically:*

$$\text{sign}(d(p_1, \ell_3)) \cdot \text{sign}(d(p_2, \ell_3)) < 0,$$

$$\text{sign}(d(p_1, \ell_4)) \cdot \text{sign}(d(p_2, \ell_4)) < 0.$$

4. *The points q_1 and q_2 lie on opposite sides of the lines ℓ_1 and ℓ_2,specifically:*

$$\text{sign}(d(q_1, \ell_1)) \cdot \text{sign}(d(q_2, \ell_1)) < 0,$$

$$\text{sign}(d(q_1, \ell_2)) \cdot \text{sign}(d(q_2, \ell_2)) < 0.$$

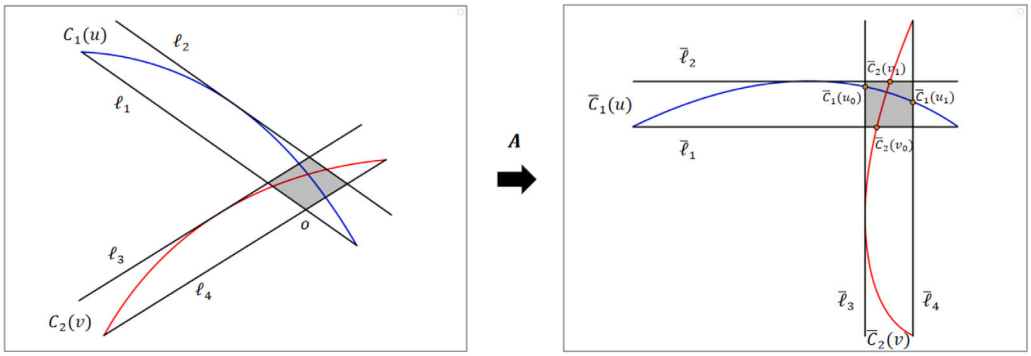


Fig. 4. Curves before and after transformation.

Proof. The proof is established by mapping the general geometric configuration into a Cartesian coordinate system and subsequently applying the Poincaré–Miranda Theorem [31].

Let L_p and L_q be two lines intersecting at point $o \in \mathbb{R}^2$. We define an orthonormal transformation $\mathbf{A}$ (comprising rotation and translation) such that in the transformed coordinate system (X, Y), the line L_p is parallel to the X-axis and L_q is parallel to the Y-axis. For any point $(x, y)^T$ in the original space, the transformed coordinates are given by:

$$(X, Y)^T = \mathbf{A}(x - o_x, y - o_y)^T.$$

Under this transformation, the four boundary lines $\ell_1, \ell_2, \ell_3, \ell_4$ become parallel to the respective coordinate axes. Let the transformed curves be denoted by $\bar{C}_1(u)$ and $\bar{C}_2(v)$. Let the transformed boundary lines be denoted by $\bar{\ell}_1, \bar{\ell}_2, \bar{\ell}_3, \bar{\ell}_4$ (see Fig. 4 for illustration).

Consider a continuous vector-valued function $\mathbf{F}(u, v)$ defined over the rectangular domain $D = [u_0, u_1] \times [v_0, v_1]$ in the parameter space:

$$\mathbf{F}(u, v) = \bar{C}_1(u) - \bar{C}_2(v) = \begin{pmatrix} f_1(u, v) \\ f_2(u, v) \end{pmatrix},$$

where $\bar{C}_1(u) = (X_1(u), Y_1(u))^T$ and $\bar{C}_2(v) = (X_2(v), Y_2(v))^T$. The components are expressed as:

$$f_1(u, v) = X_1(u) - X_2(v),$$

$$f_2(u, v) = Y_1(u) - Y_2(v).$$

According to conditions 3 and 4 of the theorem: The endpoints p_1, p_2 of the curve C_1 lie on opposite sides of the boundary lines ℓ_3 and ℓ_4. In the transformed system, this implies that for the endpoints of the parameter $u \in \{u_0, u_1\}$, the first component f_1 maintains opposite signs regardless of v:

$$\text{sign}(f_1(u_0, v)) \cdot \text{sign}(f_1(u_1, v)) < 0, \quad \forall v \in [v_0, v_1].$$

Similarly, the endpoints q_1, q_2 of the curve C_2 lie on opposite sides of ℓ_1 and ℓ_2. This ensures that for $v \in \{v_0, v_1\}$, the second component f_2 exhibits opposite signs regardless of u:

$$\text{sign}(f_2(u, v_0)) \cdot \text{sign}(f_2(u, v_1)) < 0, \quad \forall u \in [u_0, u_1].$$

The Poincaré–Miranda Theorem states that for a continuous mapping $\mathbf{F} : D \subset \mathbb{R}^n \to \mathbb{R}^n$, if each component f_i takes opposite signs on the opposite faces of the n-dimensional hyperrectangle, there exists at least one point $\mathbf{x}^* \in D$ such that $\mathbf{F}(\mathbf{x}^*) = \mathbf{0}$. For the case $n = 2$, there must exist a pair of parameter $(u^*, v^*) \in D$ such that:

$$\bar{C}_1(u^*) = \bar{C}_2(v^*).$$

Since the transformation $\mathbf{A}$ is a homeomorphism that preserves topological intersections, it follows that the original curves C_1 and C_2 possess at least one intersection point in $\mathbb{R}^2$. □

In verifying Theorem 2, it is necessary to determine whether two points lie on opposite sides of a line. To determine if two points p_1 and p_2 lie on opposite sides of a line ℓ:

1. Calculate the signed distances $d_1 = d(p_1, \ell)$ and $d_2 = d(p_2, \ell)$. The sign of the distance indicates which side of the line the point is on:

   - Positive (+): one side,
   - Negative (−): the other side,

- Zero (0): exactly on the line.

2. Check the product of their signs:

- If $\text{sign}(d_1) \cdot \text{sign}(d_2) < 0$, the points are on opposite sides → return True.
- If $\text{sign}(d_1) \cdot \text{sign}(d_2) \geq 0$, they are on the same side or at least one is on the line → return False.

We write the above algorithm as a function $flag = ArePointsOnOpposite Sides(p_1, p_2, \ell)$. Next, we will give an algorithm for a complete intersection existence criterion .

Algorithm 1 Criterion for the existence of an intersection point

Require: Two NURBS curves $C_1(u), C_2(v)(u \in [a_1, b_1], v \in [a_2, b_2])$.
Ensure: A Boolean value $flag$,*true* represents the existence of an intersection in the given area, while $false$ represents uncertainty regarding the existence of an intersection.

Extract endpoints:$p_1 \leftarrow C_1(a_1), p_2 \leftarrow C_1(b_1)$,
    $q_1 \leftarrow C_2(a_2), q_2 \leftarrow C_2(b_2)$
Compute base lines: $L_p \leftarrow \overline{p_1 p_2}, L_q \leftarrow \overline{q_1 q_2}$
$\ell_1, \ell_2 = ComputeSupportLines(C_1)$,
    $\ell_3, \ell_4 = ComputeSupportLines(C_2)$
if $C_1$is not monotonic **or** $C_2$is not monotonic **then**
    return false
end if
if $L_p \cap L_q = \emptyset$ **then**
    return false
end if
if $ArePointsOnOppositeSides(p_1, p_2, \ell_3)$ **and**
    $ArePointsOnOppositeSides(p_1, p_2, \ell_4)$ **and**
    $ArePointsOnOppositeSides(q_1, q_2, \ell_1)$ **and**
    $ArePointsOnOppositeSides(q_1, q_2, \ell_2)$ **then**
    return true
else
    return false
end if

3.3. Criterion for the uniqueness of an intersection point based on tangent cones

Sederberg [32] proposed a theoretical criterion for determining whether two planar Bézier curves have at most one intersection. We extend this theory to the case of planar NURBS curves. We adopt the method from [29] to calculate the tangent cones of the NURBS curves. In subsequent calculations, since the magnitudes of the tangent vectors are not required, we define T to represent the tangent cone of curve C, specifically:

$$T = \left\{ \lambda \frac{dC}{du} \middle| \, u \in [a, b], \lambda > 0 \right\},$$

where $\frac{dC}{du}$ denotes the tangent vector of the curve C at the parameter u.

Theorem 3. *If the tangent cones of two curves do not overlap, specifically:*

$$T_1 \cap T_2 = \emptyset.$$

Then, the two curves have at most one intersection point.

Proof. Assume that C_1 and C_2 have two distinct intersection points P and Q, that is, there exist parameters $u_1, u_2 \in [a_1, b_1]$ and $v_1, v_2 \in [a_2, b_2]$, such that:

$$C_1(u_1) = C_2(v_1) = P,$$

$$C_1(u_2) = C_2(v_2) = Q,$$

$$P \neq Q.$$

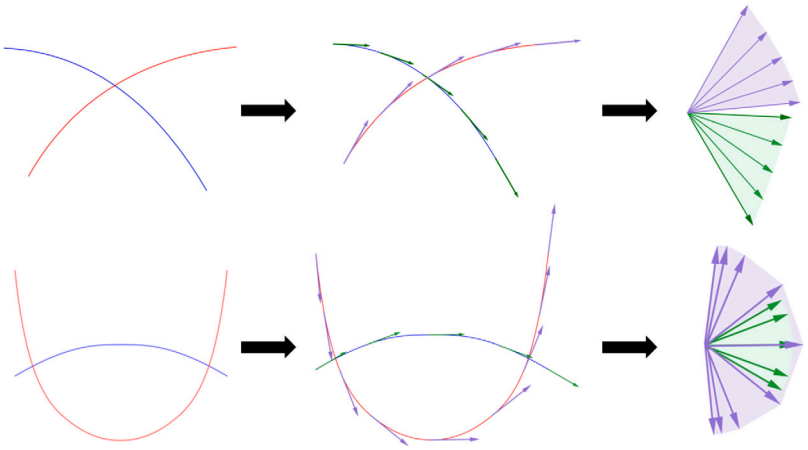


Fig. 5. Comparative illustration of tangent cones for distinct intersection cases.

For the segment of C_1 from P to Q, the parameterization is defined as $\alpha(t) = C_1(u_1 + t(u_2 - u_1))$, where $t \in [0, 1]$. At this point:

$$\alpha(0) = P, \alpha(1) = Q,$$

$$\alpha'(t) = (u_2 - u_1) \cdot \frac{dC_1}{du}(u_1 + t(u_2 - u_1)) \in T_1.$$

Similarly, for C_2, the parameterization is defined as $\beta(t) = C_2(v_1 + t(v_2 - v_1))$, then:

$$\beta(0) = P, \beta(1) = Q,$$

$$\beta'(t) = (v_2 - v_1) \cdot \frac{dC_2}{dv}(v_1 + t(v_2 - v_1)) \in T_2.$$

Define the function $f(t) = \alpha(t) - \beta(t)$. Then $f(0) = f(1) = 0$. f is continuous on $[0, 1]$ and differentiable on $(0, 1)$. According to Rolle's Theorem [33], there exists some $t_0 \in (0, 1)$ such that:

$$f'(t_0) = \alpha'(t_0) - \beta'(t_0) = 0 \Longrightarrow \alpha'(t_0) = \beta'(t_0).$$

Given that $\alpha'(t_0) \in T_1$ and $\beta'(t_0) \in T_2$, it follows that $\alpha'(t_0) = \beta'(t_0) \in T_1 \cap T_2$. This contradicts the assumption that $T_1 \cap T_2 = \emptyset$. Therefore, the assumption that C_1 and C_2 have two distinct intersection points does not hold, and hence they have at most one intersection point. □

As illustrated in Fig. 5, the tangent cone structures of two planar curves are illustrated under two distinct intersection scenarios. The upper part depicts the tangent cones of two curves with a single intersection point, while the lower part illustrates those of two curves with two intersection points.

3.4. Singular case

While the primary contribution of our proposed algorithm lies in the efficient handling of general (non-singular) cases, addressing singular configurations is essential for algorithmic completeness. For special intersection cases, such as tangency, cusps, self-intersections, or intersections at endpoints (see Fig. 6), we employ established numerical methods. These standard techniques are integrated solely to ensure the algorithm's robustness and capability to cover all possible intersection scenarios.

For tangent curves, we construct a system of equations:

$$\begin{cases} C_1(u) = C_2(v) \\ \frac{dC_1(u)}{du} = \frac{dC_2(v)}{dv}. \end{cases}$$

The tangential point is solved by transforming the original system of equations into an optimization problem that minimizes the sum of squared residual norms, which is then solved using the Levenberg–Marquardt [34,35] algorithm .

For cases where the intersection point coincides with a self-intersection, we examine the computed solutions to determine whether two distinct parameter values correspond to the same point within the specified tolerance. Specifically, for the case where C_1 intersects C_2 at a self-intersection point of C_2, the following condition holds:

$$\begin{cases} u_1 = u_2 \\ C_1(u_1) = C_1(u_2) = C_2(v_1) = C_2(v_2), \end{cases}$$

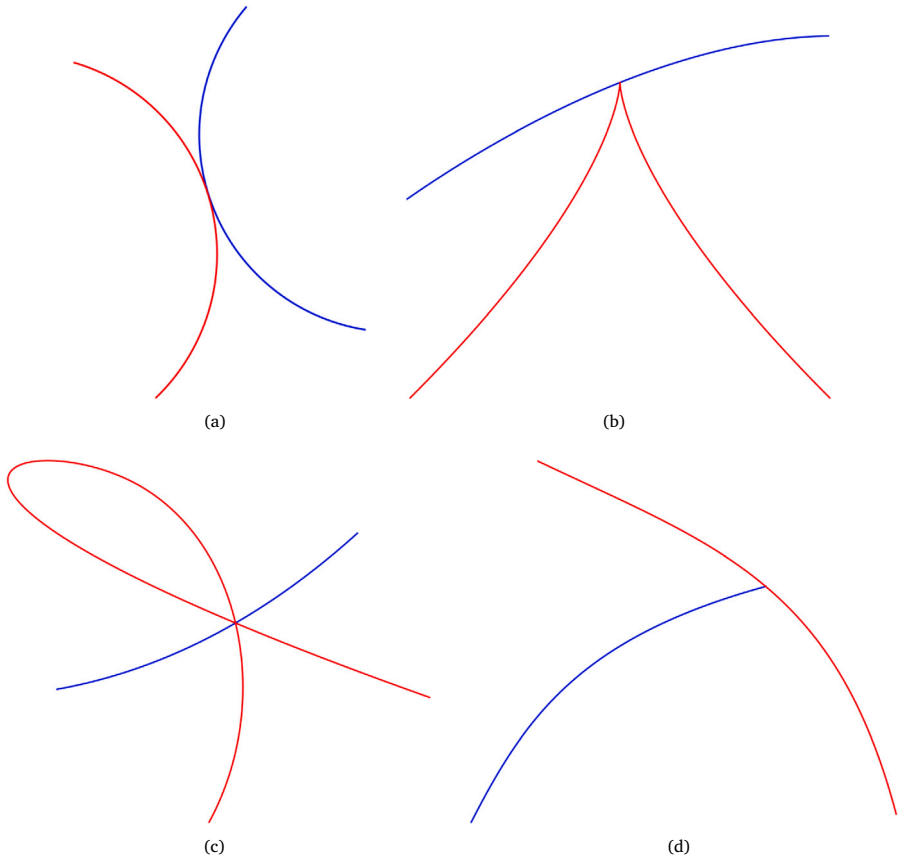


Fig. 6. Several singular intersection cases. (a) Two tangent curves. (b) A cusped curve intersects another at the cusp. (c) A self-intersecting curve intersects another at the self-intersection. (d) Intersection at a curve's endpoint.

where u_1 and v_1 are the computed parameter values of one intersection point, corresponding to C_1 and C_2, respectively; u_2 and v_2 represent another set. The same principle applies to the case where C_2 intersects a self-intersection point of C_1.

For cases where the intersection point lies at a cusp, we determine the existence of a cusp by computing the tangent vector. Specifically, if

$$\left.\frac{dC_1}{du}\right|_{u=u_0} = 0,$$

then the curve C_1 has a cusp at the parameter value u_0. The same principle applies to the curve C_2.

For cases where intersections occur at curve endpoints, we determine whether the intersection point lies on the other curve's endpoint by checking the distance between the intersection point and the endpoint. Specifically, for the point at the parameter value u on curve C_1 intersecting with the endpoints of curve C_2 within precision tolerance:

$$|C_1(u) - C_2(a_2)| < \epsilon \quad \vee \quad |C_1(u) - C_2(b_2)| < \epsilon,$$

then, this indicates that curve C_1 intersects with an endpoint of C_2 at parameter value u within the specified tolerance ϵ. The same principle applies to the case where C_2 intersects with an endpoint of C_1.

The detection of tangency points is handled during the singularity processing phase. After the intersection points are computed, they are examined to determine if they constitute a singular case, such as a cusp, self-intersection, or endpoint. Notably, in these singular scenarios, intersecting points obtained solely via numerical methods are highly precision dependent. The existence of an intersection can only be asserted within a certain tolerance, rather than conclusively established as in non-singular cases.

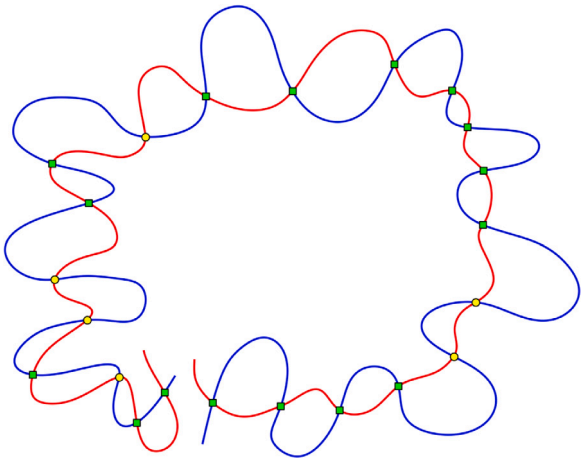


Fig. 7. The results of the Bézier Clipping method for example 4 are presented, where the yellow points represent the intersections found by the Bézier Clipping method, and the green points indicate the intersections that were missed by the Bézier Clipping method. Example 4 has 22 intersections. Bézier Clipping found 6 of them and missed 16. And our method can find all 22 intersections.

4. Numerical experiments

4.1. Experimental setup and comparative analysis

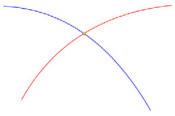
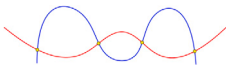
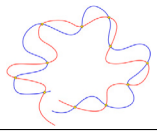
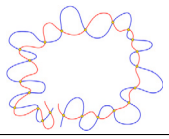
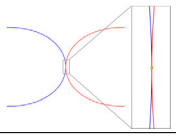
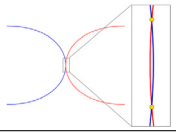
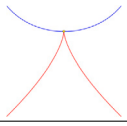
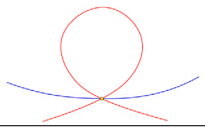
In this section, we will present several examples to illustrate our algorithm for computing the intersection points of two NURBS curves. Our algorithm was implemented in C++. All experiments were conducted on a computer equipped with an Intel Core i7-14700HX processor and 32 GB of RAM. In the experiments of this section, the convergence tolerance for the Newton iteration was set to 10^{-8}.

We selected eight examples to comprehensively evaluate the algorithm. Example 1 consists of two simple NURBS curves with one intersection. Example 2 consists of two simple NURBS curves with four intersections. Examples 1 and 2 represent simple cases of curve–curve intersection. Examples 3 and 4 consist of complex NURBS curves with high-degree polynomials and multiple control points that have several intersections. They represent high-degree, multi-control-point cases of curve–curve intersection. Example 5 consists of two tangent NURBS curves with one intersection. Example 6 consists of two nearly tangent curves with two intersections, where the distance between the two intersection points is greater than 10^{-8}. Example 7 consists of a curve with a cusp intersecting another curve, with the intersection point located at the cusp. Example 8 consists of a self-intersecting curve intersecting another curve, with the intersection point located at the self-intersection. Examples 5, 6, 7, and 8 represent singular cases of curve–curve intersection.

We conducted comprehensive comparisons with multiple classical methods in terms of both computational efficiency and accuracy. The Bézier Clipping and Geometry Interval were implemented as the classical Bézier curve intersection algorithm. Given that all experimental cases utilized NURBS curves, the input NURBS curves were segmented into Bézier curves for the intersection computation, where the segmentation time was included in the total intersection computation time. SISL (The SINTEF Spline Library) [36] and OCC (Open Cascade) [37] are well-known geometric modeling libraries that provide the function of curve intersection finding. SISL and OCC both employ a combination of subdivision and Newton iteration to compute the intersection points of spline curves. The primary difference lies in their stopping criteria for subdivision: OCC uses the condition that the bounding box distance is smaller than the parametric tolerance, whereas SISL introduces an additional linearization threshold for the curves. Specifically, SISL halts the subdivision process not only when the bounding box distance falls below a certain threshold, but also when the curves are subdivided to the extent that they can be approximated as straight lines. Intersections are computed using the *IntTools_EdgeEdge* class in OCC and the *s1857* function in SISL. The numerical tolerance for both methods is set to 10^{-8}.

The comprehensive experimental results are presented in Table 1. The experimental results show that the Bézier Clipping method may miss solutions when dealing with singular cases and high-order complex curves. In the case of example 4, the intersection points missed by the Bézier Clipping method are displayed in Fig. 7. SISL may produce incorrect results when dealing with singular cases. In Fig. 8, we plot the distance relationship between the two curves in Examples 7 and 8. SISL misses an intersection in both cases. Although OCC can accurately find all intersections, their computation time rises sharply with curve complexity. In contrast, our method quickly finds all intersections with stable efficiency, showing advantages in both speed and result completeness. We illustrate the computational process of our method in Example 2 in Fig. 9.

Table 1
The experimental results of our method are compared with the existing methods. The Curve Info encompasses the number of control points, order of curves, and intersection counts. The data in bold indicates the best performance.

Example	NURBS curves	Curve info	Methods	Time(μs)
Example 1		C_1 : 3/3/1 C_2 : 3/3/1	Bézier Clipping Geometry Interval SISL OCC Our Method	134 2075 108 1432 **90**
Example 2		C_1 : 8/4/4 C_2 : 6/4/4	Bézier Clipping Geometry Interval SISL OCC Our Method	921 4628 591 2108 **195**
Example 3		C_1 : 59/8/11 C_2 : 44/8/11	Bézier Clipping Geometry Interval SISL OCC Our Method	7734 5803 33020 18342 **1076**
Example 4		C_1 : 100/8/22 C_2 : 100/8/22	Bézier Clipping Geometry Interval SISL OCC Our Method	42862[a] 20839 60356 42438 **2052**
Example 5		C_1 : 4/4/1 C_2 : 4/4/1	Bézier Clipping Geometry Interval SISL OCC Our Method	969[a] 1563 841 2804 **306**
Example 6		C_1 : 4/4/2 C_2 : 4/4/2	Bézier Clipping Geometry Interval SISL OCC Our Method	866[a] 1237[a] 1973 4246 **391**
Example 7		C_1 : 4/4/1 C_2 : 4/4/1	Bézier Clipping Geometry Interval SISL OCC Our Method	1177[a] 2976[a] 1123[a] 875 **291**
Example 8		C_1 : 5/4/2 C_2 : 9/4/2	Bézier Clipping Geometry Interval SISL OCC Our Method	1442[a] 3892[a] 3275[a] 1312 **544**

[a] Indicates missing intersections.

4.2. Algorithmic efficiency and scalability analysis

For non-singular cases, we examine the computational time and frequency associated with key procedural steps. As summarized in Table 2, the number of subdivisions executed for each curve — which also reflects the number of bounding volume constructions — is reported. Furthermore, we document the total number of evaluations, along with the average time per evaluation, for both the criterion that assesses the existence of an intersection point and the criterion that verifies its uniqueness.

We evaluated the computational performance of the algorithm using Example 3 and Example 4, employing the knot insertion method to increase the number of control points from 100 to 1000 while strictly preserving the original curve geometry. Experimental results demonstrate that, despite the increasing number of control points, the algorithm consistently and accurately determined the number of intersection points without deviation. The computational times corresponding to varying numbers of control points are illustrated in Fig. 10.

In order to improve the validation of the algorithm's reliability, we augmented the initial test cases in Table 1 with more challenging geometric models. The extended set encompasses complex geometric models characterized by significant scale discrepancies, high-density intersections (numbering in the hundreds), and multiple tangential contacts. These instances were primarily generated through the stochastic synthesis of control points, a method that effectively simulates diverse topological structures and thus provides a rigorous environment for assessing the algorithm's robustness under extreme conditions. The intersection results for these complex

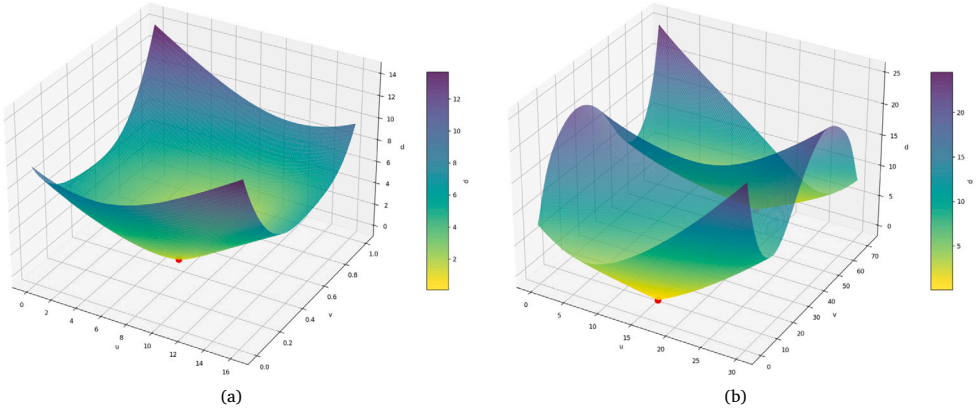


Fig. 8. The distance plots for the two curves in Example 7 and Example 8. (A) and (B) correspond to Example 7 and Example 8 respectively. The x and y axes represent parameter values, while the z axis shows distance. Darker colors indicate bigger distances, with red points marking where the distance is zero.

Table 2
Execution Time and Number of Executions for Principal Steps in Non-Singular Scenarios. The Subdivision denotes the number of subdivision operations performed by C_1 and C_2. The Exist indicates the number of times the criterion for the existence of an intersection point is evaluated. The Exist Time indicates the average time consumed per evaluation of the criterion for the existence of an intersection point, measured in μs. The Unique indicates the number of times the criterion for the uniqueness of an intersection point is evaluated. The Unique Time indicates the average time consumed per evaluation of the criterion for the uniqueness of an intersection point, measured in μs.

Example	Subdivision	Exist	Exist time	Unique	Unique time
Example 1	0/0	1	2.90	1	3.10
Example 2	3/3	7	0.82	6	0.88
Example 3	85/77	81	1.01	23	1.12
Example 4	177/143	159	1.06	36	1.18

scenarios are illustrated in Fig. 11. In the legend, the numerical values enclosed in parentheses following the term "Intersections" denote the total count of detected intersection points. These results demonstrate that the proposed algorithm maintains robust performance when handling complex cases, consistently yielding accurate solutions across all instances.

5. Applications

5.1. 2D closed curves boolean operation

Boolean operations on closed 2D curves necessitate the clear establishment of the interior and exterior regions of each operand. To perform the operations (e.g., Union, Difference, Intersection), it is critically required to accurately and robustly compute the intersection points between the two boundary curves.

As demonstrated in Fig. 12, the intersection calculation is the pivotal step. The Minuend curve (the blue ring) and the Subtrahend curve (the red circle) are analyzed to locate all points where they cross (bottom right). The precise determination of these intersection points, which are highlighted by the green markers in the figure, provides the critical loci that define the segments of the output boundary.

Failure to accurately compute these intersections, particularly in cases involving tangency or high-order continuity at the crossing points, leads to topological inconsistencies and numerical instability. By effectively computing the intersections between the two closed curves, the algorithm can precisely determine the segments of the original boundaries that form the final result (bottom left), thereby ensuring the correct and consistent outcome of the 2D Boolean operation.

5.2. Surface trimming

The process of trimming a spline surface requires precisely defining the effective domain of the surface by using trimming curves. These trimming curves are usually represented as 2D spline curves residing in the surface's parameter space (the uv-plane).

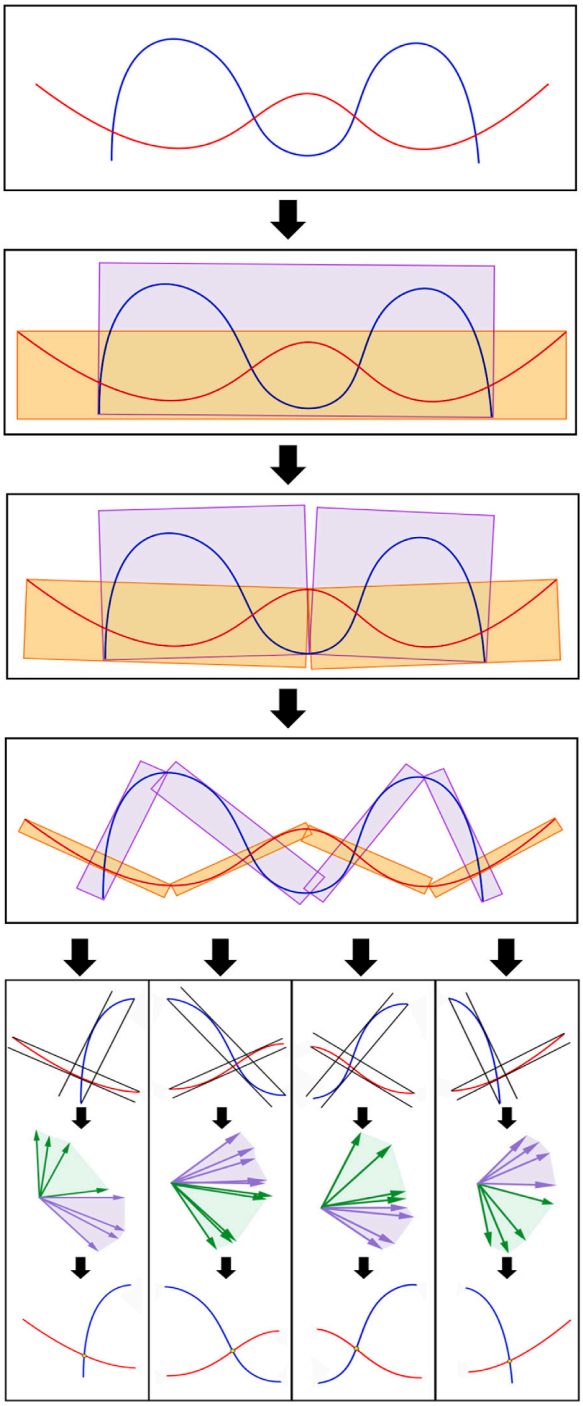


Fig. 9. Computational process of Example 2. Initially, the algorithm uses OBB to determine intersections. For intersecting regions, it checks existence; if unsatisfied, curves are subdivided. For regions confirmed to exist, compute their tangent cones. If normal cones are disjoint, apply Newton iteration to find intersections. The figure only shows regions with solutions.

As illustrated in Fig. 13, the original surface (top left) corresponds to a rectangular domain in the uv-plane (top center), which already contains an initial trimming loop. To successfully complete the trimming operation with a new boundary, it is essential to accurately and robustly compute the intersections among the different trimming curves, as well as the intersections between the trimming curves and the natural boundaries of the parameter domain. The newly added trimming curve is shown in the uv-plane

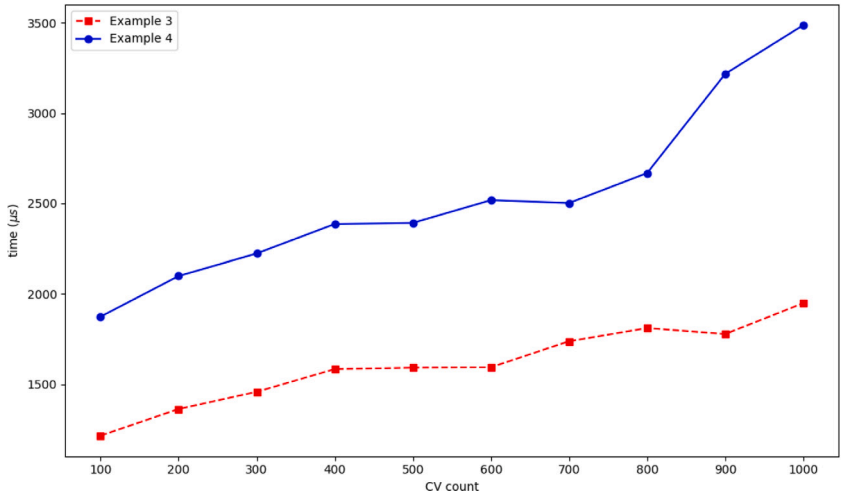


Fig. 10. Evaluation of computational efficiency versus the number of control points under fixed curve geometry.

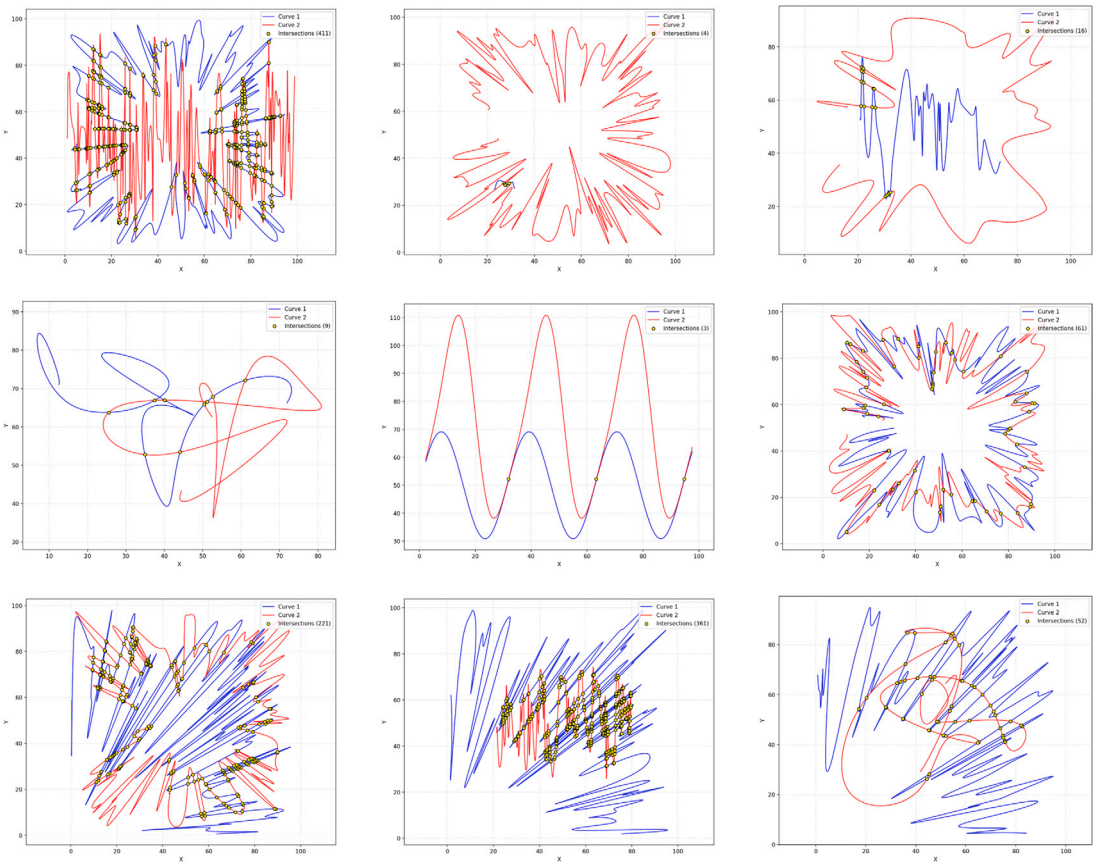


Fig. 11. Intersection results for complex geometric cases.

(top right, marked in red). These intersection points are fundamental, as they define the vertices of the loops and regions that constitute the final valid parameter domain (bottom right). Failure to accurately determine these 2D intersection points in the *uv*-plane, particularly those involving tangential contact or near-singular configurations, results in gaps or overlaps in the resulting surface's definition, thereby compromising the geometric and topological integrity of the final trimmed surface (bottom left).

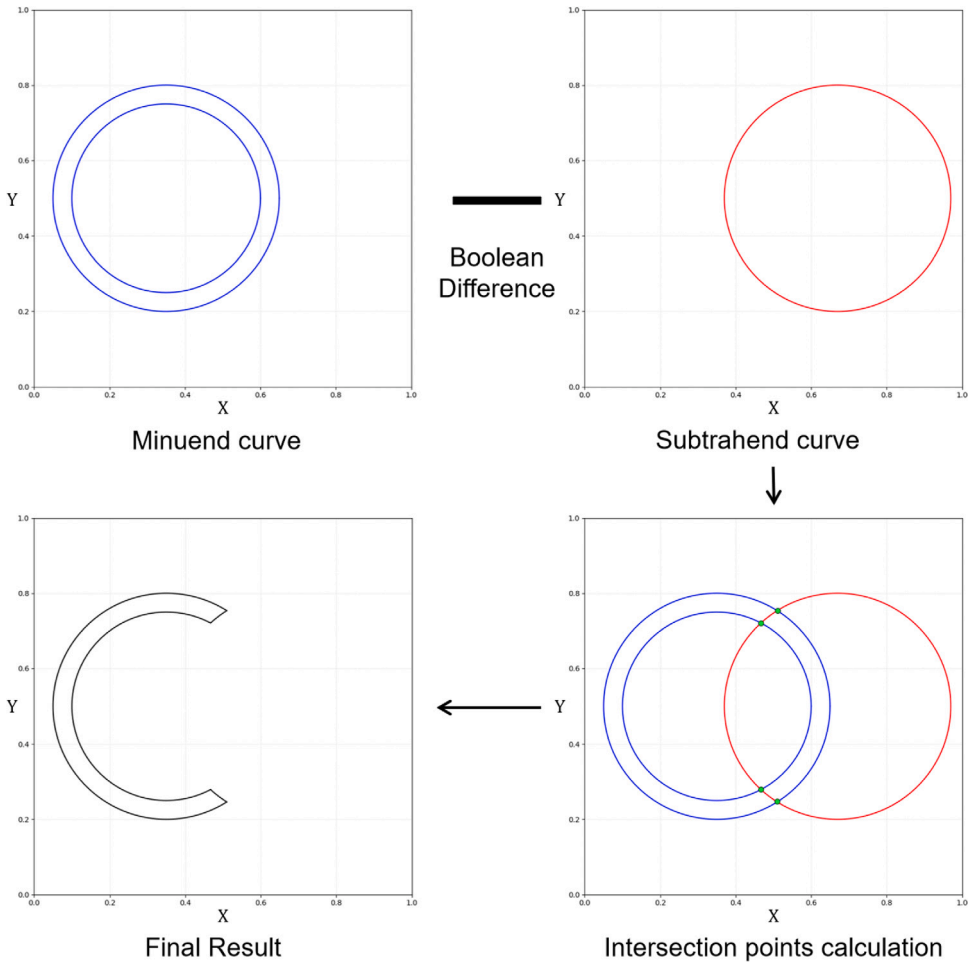


Fig. 12. Application of 2D spline curve intersection for a Boolean Difference operation (Minuend - Subtrahend). The green markers denote the accurately computed intersection points that are used to define the final C-shaped boundary.

6. Conclusions

In this paper, we have developed a fast and robust algorithm to address the challenge of NURBS curve intersection. By utilizing the intersection existence theorem proposed and the Hodograph theory, we have established a method to determine the number of intersection points within a specified region. After obtaining the interval where a unique intersection point exists, we employ the Newton iteration method to converge rapidly to the intersection point. Experimental results demonstrate that our algorithm outperforms classical and existing methods in terms of robustness and computational efficiency, and it can directly calculate the intersection points of NURBS curves without decomposing them into Bézier curve segments.

The algorithm is primarily designed for computing intersection points between two-dimensional NURBS curves, but it cannot process coincident curve cases. Furthermore, it lacks the capability to handle cases involving coincident curves. Future work will focus on extending the current method to three-dimensional spline curve intersections and surface–surface intersection problems.

Funding

This work was partially supported by the National key R&D plan [No. 2022YFC3802102].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

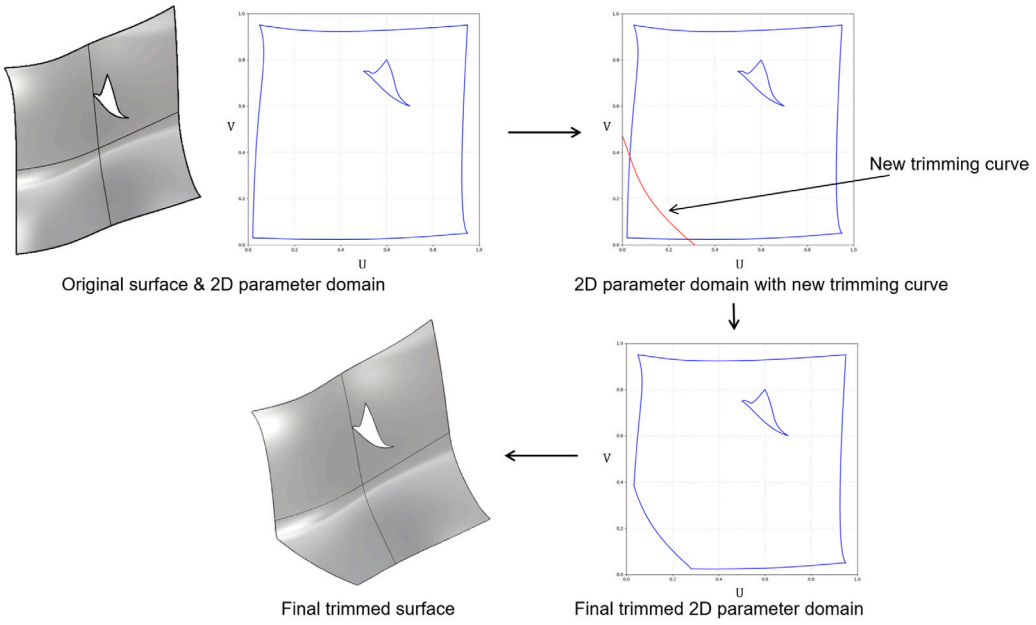


Fig. 13. Illustration of the surface trimming process in the parameter domain.

Data availability

The data that has been used is confidential.

References

[1] Bézier P. Numerical control-mathematics and applications. Transl By AR Forrest 1972.
[2] Hoffmann CM. Geometric and solid modeling : an introduction. 1989, Geometric and solid modeling : an introduction.
[3] Patrikalakis NM, Maekawa T. In: Shape interrogation for computer aided design and manufacturing, vol. 15, Springer; 2002.
[4] Ji Y, Yu Y-Y, Wang M-Y, Zhu C-G. Constructing high-quality planar NURBS parameterization for isogeometric analysis by adjustment control points and weights. J Comput Appl Math 2021;396:113615.
[5] Gordon WJ, Riesenfeld R. B-spline curves and surfaces, in computer aided geometric design. In: Proc. of utah conference. new york: Academic Press; 1974.
[6] Gordon WJ, Riesenfeld RF. B-spline curves and surfaces. In: Computer aided geometric design. Elsevier; 1974, p. 95–126.
[7] Riesenfeld RF. Applications of b-spline approximation to geometric problems of computer-aided design.. Syracuse University; 1973.
[8] Versprille KJ. Computer-aided design applications of the rational b-spline approximation form.. Syracuse University; 1975.
[9] Tiller W. Rational B-splines for curve and surface representation. IEEE Comput Graph Appl 1983;3(06):61–9.
[10] Xu J, Yap C. Effective subdivision algorithm for isolating zeros of real systems of equations, with complexity analysis. In: Proceedings of the 2019 international symposium on symbolic and algebraic computation. ISSAC '19, New York, NY, USA: Association for Computing Machinery; 2019, p. 355–62. http://dx.doi.org/10.1145/3326229.3326270.
[11] Cheng J-S, Wen J, Zhang B. Certified numerical real root isolation for bivariate nonlinear systems. J Symbolic Comput 2023;114:149–71. http://dx.doi.org/10.1016/j.jsc.2022.04.005, URL https://www.sciencedirect.com/science/article/pii/S0747717122000256.
[12] Tan L, Li B, Zhang B, Cheng J-S. An algorithm for the intersection problem of planar parametric curves. In: International workshop on computer algebra in scientific computing. Springer; 2023, p. 312–29.
[13] Limaiem A, Trochu F. Geometric algorithms for the intersection of curves and surfaces. Comput Graph 1995;19(3):391–403. http://dx.doi.org/10.1016/0097-8493(95)00009-2, URL https://www.sciencedirect.com/science/article/pii/0097849395000092.
[14] Sederberg T, Nishita T. Curve intersection using Bézier clipping. Computer-Aided Des 1990;22(9):538–49. http://dx.doi.org/10.1016/0010-4485(90)90039-F, URL https://www.sciencedirect.com/science/article/pii/001044859090039F.
[15] Schulz C. Bézier clipping is quadratically convergent. Comput Aided Geom Design 2009;26(1):61–74. http://dx.doi.org/10.1016/j.cagd.2007.12.006, URL https://www.sciencedirect.com/science/article/pii/S0167839607001434.
[16] Bartoň M, Jüttler B. Computing roots of polynomials by quadratic clipping. Comput Aided Geom Design 2007;24(3):125–41. http://dx.doi.org/10.1016/j.cagd.2007.01.003, URL https://www.sciencedirect.com/science/article/pii/S0167839607000106.
[17] Liu L, Zhang L, Lin B, Wang G. Fast approach for computing roots of polynomials using cubic clipping. Comput Aided Geom Design 2009;26(5):547–59. http://dx.doi.org/10.1016/j.cagd.2009.02.003, URL https://www.sciencedirect.com/science/article/pii/S0167839609000338.
[18] Lou Q, Liu L. Curve intersection using hybrid clipping. Comput Graph 2012;36(5):309–20. http://dx.doi.org/10.1016/j.cag.2012.03.021, URL https://www.sciencedirect.com/science/article/pii/S009784931200060X. Shape Modeling International (SMI) Conference 2012.
[19] North NS. Brigham Young University; 2007.
[20] Yuan Q. Study on hybrid clipping method for solving polynomial roots (Ph.D. thesis), China: Zhejiang University Hangzhou; 2012.
[21] Wu Y, Li X. Curve intersection based on cubic hybrid clipping. Vis Comput Ind Biomed Art 2022;5(1):17. http://dx.doi.org/10.1186/s42492-022-00114-3, URL https://vciba.springeropen.com/articles/10.1186/s42492-022-00114-3.

[22] Shao W, Chen F, Liu X. Robust algebraic curve intersections with tolerance control. Computer-Aided Des 2022;147:103236. http://dx.doi.org/10.1016/j.cad.2022.103236, URL https://www.sciencedirect.com/science/article/pii/S001044852200029X.
[23] Yu Y-Y, Li X, Ji Y. On intersections of B-spline curves. Mathematics 2024;12(9). http://dx.doi.org/10.3390/math12091344, URL https://www.mdpi.com/2227-7390/12/9/1344.
[24] Beccari CV, Casciola G. A cox-de boor-type recurrence relation for C1 multi-degree splines. Comput Aided Geom Design 2019;75:101784. http://dx.doi.org/10.1016/j.cagd.2019.101784, URL https://www.sciencedirect.com/science/article/pii/S0167839619300937.
[25] Tiller W. Rational B-Splines for Curve and Surface Representation . IEEE Comput Graph Appl 1983;3(06):61–9. http://dx.doi.org/10.1109/MCG.1983.263244, URL https://doi.ieeecomputersociety.org/10.1109/MCG.1983.263244.
[26] Piegl LA, Tiller W. The NURBS book. 2nd ed.. Berlin, Heidelberg: Springer-Verlag; 1997.
[27] Clark JH. Hierarchical geometric models for visible surface algorithms. Commun ACM 1976;19(10):547–54. http://dx.doi.org/10.1145/360349.360354.
[28] Sederberg TW, Wang X. Rational hodographs. Comput Aided Geom Design 1987;4(4):333–5. http://dx.doi.org/10.1016/0167-8396(87)90008-2, URL https://www.sciencedirect.com/science/article/pii/0167839687900082.
[29] Saito T, Wang G-J, Sederberg TW. Hodographs and normals of rational curves and surfaces. Comput Aided Geom Design 1995;12(4):417–30. http://dx.doi.org/10.1016/0167-8396(94)00023-L, URL https://www.sciencedirect.com/science/article/pii/016783969400023L.
[30] De Boor C. On calculating with B-splines. J Approx Theory 1972;6(1):50–62.
[31] Kulpa W. The Poincaré-Miranda Theorem. The American Mathematical Monthly 1997;104(6):545–50, http://www.jstor.org/stable/2975081.
[32] Sederberg TW, Meyers RJ. Loop detection in surface patch intersections. Comput Aided Geom Design 1988;5(2):161–71. http://dx.doi.org/10.1016/0167-8396(88)90029-5, URL https://www.sciencedirect.com/science/article/pii/0167839688900295.
[33] Fiorenza A, Fiorenza R. Generalizations of Rolle's theorem. Mathematics 2024;12(14):2157.
[34] Levenberg K. A method for the solution of certain non-linear problems in least squares. Quart Appl Math 1944;2(2):164–8. http://dx.doi.org/10.1090/qam/10666.
[35] Marquardt DW. An algorithm for least-squares estimation of nonlinear parameters. J Soc Ind Appl Math 1963;11(2):431–41. http://dx.doi.org/10.1137/0111030.
[36] SINTEF ICT Gg. The SINTEF spline library. 2025, https://github.com/SINTEF-Geometry/SISL.
[37] Open Cascade SAS. Open CASCADE technology (OCCT). 1999-2023, URL https://www.opencascade.com/. 3D modeling and numerical simulation software development toolkit.